# EXPERIMENT 5

**Student Name: ADITYA MANHAS** **UID: 24BAI70883**
**Branch: AIT-CSE-AIML** **Section/Group: 24AIT_KRG-G2**
**Semester: 4**
**Subject Name: Database Management System** **Subject Code: 24CSH-298**

## Experiment 5 – SQL Conditional Logic (Odd & Even Values)

SQL Conditional Logic using the MOD (%) operator to classify employee salary values as Odd or Even. This experiment demonstrates numerical analysis and conditional evaluation using SQL queries in Oracle and PostgreSQL.

## Aim

To understand and apply conditional logic in SQL by using the modulus operator (MOD / %) to analyze numerical data and classify employee salaries as odd or even, thereby improving data analysis and decision-making skills in SQL.

## Objective

- To write and execute SQL queries using the MOD (%) operator.
- To retrieve salary details from an employee table.
- To classify salaries into Odd and Even categories.
- To display odd and even salary values separately.

## Software Requirements

Database Management System:
Oracle Database Express Edition (Oracle XE)
PostgreSQL Database

Database Administration Tool / Client Tool:
Oracle SQL Developer (for Oracle XE)
pgAdmin (for PostgreSQL)

# EXPERIMENT 5

## Practical / Experiment Steps

1. Create an employee table with columns:
   id
   name
   department
   salary
2. Insert sample employee salary records into the table.
3. Write a SQL query to retrieve employee salary details.
4. Use the MOD(salary, 2) function (Oracle) or salary % 2 (PostgreSQL) to check whether the salary is odd or even.
5. Use a CASE statement to classify salaries as:
   'Odd Salary'
   'Even Salary'
6. Execute separate queries to:
   Display only employees with Odd salaries.
   Display only employees with Even salaries.
7. Observe and record the output results.

## Procedure

1. Open Oracle SQL Developer or pgAdmin.
2. Connect to the database.
3. Create the employee table using CREATE TABLE.
4. Insert sample records using INSERT INTO.
5. Execute SELECT queries with the MOD operator.
6. Apply CASE statement for conditional classification.
7. Display results and capture screenshots.

## Input / Output Details

### Input

Table Name: employee

# EXPERIMENT 5

**Columns:**
id (Integer)
name (Varchar)
department (Varchar)
salary (Integer)

**Logic Used:**
MOD(salary, 2) or salary % 2
CASE statement

**Step-wise Output**

Step 1 – Create employee Table

Query  Query History

```sql
CREATE TABLE employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(100),
    salary INT
);
```

Data Output  Messages  Notifications

```
CREATE TABLE

Query returned successfully in 201 msec.
```

# EXPERIMENT 5

Step 2 – Insert Data into employee Table

Experiment_3/post... × Experiment-3.sql × DBMS1/postgres... × DBMS1/postgres... × exp-5/postgres@PostgreS

exp-5/postgres@PostgreSQL 18

No limit

Query Query History

```sql

INSERT INTO EMPLOYEE VALUES (101, 'Amit', 25000);
INSERT INTO EMPLOYEE VALUES (102, 'Riya', 32001);
INSERT INTO EMPLOYEE VALUES (103, 'Rahul', 45000);
INSERT INTO EMPLOYEE VALUES (104, 'Sneha', 28999);
INSERT INTO EMPLOYEE VALUES (105, 'Arjun', 50000);
```

Data Output Messages Notifications

```
INSERT 0 1

Query returned successfully in 248 msec.
```

Total rows: Query complete 00:00:00.248

# EXPERIMENT 5

Step 3 – Display Employee Salary Records

Query | Query History

```
SELECT * FROM EMPLOYEE;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5

| | emp_id [PK] integer | emp_name character varying (100) | salary integer |
|---|---|---|---|
| 1 | 101 | Amit | 25000 |
| 2 | 102 | Riya | 32001 |
| 3 | 103 | Rahul | 45000 |
| 4 | 104 | Sneha | 28999 |
| 5 | 105 | Arjun | 50000 |

Successfully run. Total

Total rows: 5 | Query complete 00:00:00.473

# EXPERIMENT 5

Step 4 – Classify Salaries as Odd or Even

Query Que File tory

```sql
SELECT EMP_ID,
       EMP_NAME,
       SALARY,
       CASE
           WHEN SALARY % 2 = 0 THEN 'Even Salary'
           ELSE 'Odd Salary'
       END AS SALARY_TYPE
FROM EMPLOYEE;
```

Data Output Messages Notifications

Showing rows: 1 to 5

| | emp_id [PK] integer | emp_name character varying (100) | salary integer | salary_type text |
|---|---|---|---|---|
| 1 | 101 | Amit | 25000 | Even Salary |
| 2 | 102 | Riya | 32001 | Odd Salary |
| 3 | 103 | Rahul | 45000 | Even Salary |
| 4 | 104 | Sneha | 28999 | Odd Salary |
| 5 | 105 | Arjun | 50000 | Even Salary |

Total rows: 5 Query complete 00:00:00.156

# EXPERIMENT 5

Step 5 – Display Only Odd Salaries

Query | Query History

```sql
SELECT * FROM EMPLOYEE
WHERE SALARY % 2 <> 0;
```

Data Output | Messages | Notifications

Showing rows: 1 to 2

| | emp_id [PK] integer | emp_name character varying (100) | salary integer |
|---|---|---|---|
| 1 | 102 | Riya | 32001 |
| 2 | 104 | Sneha | 28999 |

Successfully run. Total qu

Total rows: 2 | Query complete 00:00:00.181

## EXPERIMENT 5

Step 6 – Display Only Even Salaries

```
SELECT * FROM EMPLOYEE
WHERE SALARY % 2 = 0;
```

Data Output Messages Notifications

Showing rows: 1 to 3

| | emp_id [PK] integer | emp_name character varying (100) | salary integer |
|---|---|---|---|
| 1 | 101 | Amit | 25000 |
| 2 | 103 | Rahul | 45000 |
| 3 | 105 | Arjun | 50000 |

Successfully run. Tota

**Learning Outcome**

After completing this experiment, the learner will be able to:

- Understand conditional logic in SQL queries.
- Use the MOD (%) operator for numerical analysis.
- Classify numeric values into odd and even categories.
- Write efficient SQL queries for payroll data analysis.
- Apply SQL logic in real-world enterprise environments.